How Open Source Projects Release

Branching, versioning, and the mental model behind Spring's release process

- This talk gives you a **vocabulary and mental model**, not a how-to guide
- After this, you will know exactly what `7.2.0-M3` vs `7.2.0-RC1` vs `7.2.0` means
- You will understand why `7.0.x` and `6.2.x` both exist in the repo right now
- We use **Spring** as the primary case study, then compare to the broader ecosystem

What Does This Version Mean?

Five version strings from the same project — what is the difference?

```
7.2.0-SNAPSHOT    7.2.0-M1    7.2.0-RC1    7.2.0    7.2.1
```

- All five exist in the same project at different points in time
- **Two** of them are on Maven Central — which ones?
- **Three** require special repository configuration to even resolve
- Understanding the answer requires knowing the release lifecycle

> **Answer:** Only `7.2.0` (GA) and `7.2.1` (patch) are on Maven Central.
> Everything else lives at `repo.spring.io` only.

Three Ways to Manage Code

Git Flow — feature branches + develop + release branches + hotfixes

- Complex, designed for teams with long QA cycles
- Not what Spring uses

Trunk-based development — everyone commits to one branch; short-lived feature flags

- Fast deployment cadence
- Not what Spring uses either

Maintenance-branch model — one primary branch for the next generation + long-lived `N.N.x` branches per released minor

- Designed for supporting multiple versions in production simultaneously
- **This is Spring's model**

Spring's Actual Branches Right Now

```
main (7.1.0-SNAPSHOT)       ← next unreleased generation
  ↑ git merge --no-ff
7.0.x (7.0.8-SNAPSHOT)      ← current GA production line
  ↑ git merge --no-ff
6.2.x (6.2.19-SNAPSHOT)     ← previous generation, active OSS maintenance
  ↑ cherry-pick (backport)
6.1.x (6.1.22-SNAPSHOT)     ← winding down (last OSS commit June 2025)

6.0.x · 5.3.x · 5.2.x ...  ← EOL, preserved read-only forever
```

- The `.x` in `7.0.x` is a **literal string** — there is no `7.0.1` branch; patches are tags
- **`main` is not current stable** — it is the next unreleased generation
- Branches are **never deleted** — historical branches exist going back to Spring 3.x

Fix Once, Propagate Forward

Fixes go in at the **oldest affected branch**, then merge forward. Never backward.

```
6.2.x ——— commit "Fix unsafe static resource" ——— →
              ↓ git merge --no-ff
7.0.x ——————————————— Merge branch '6.2.x' ——— →
                            ↓ git merge --no-ff
main  ———————————————————————— Merge branch '7.0.x' ——— →
```

All active lines	Fix on oldest branch, merge forward
Older-only fix	Cherry-pick via backport bot (`for: backport-to-6.1.x` label)
Fixes never go backward	Merging `7.0.x` → `6.2.x` would risk introducing unreleased APIs

From First Commit to End of Life

```
SNAPSHOT → M1 → M2 → [M3] → RC1 → [RC2] → GA → patches → EOL
```

Stage	Example	Stability	Where
SNAPSHOT	`7.2.0-SNAPSHOT`	Mutable, CI build	repo.spring.io/snapshot
Milestone	`7.2.0-M1`	Fixed, API in flux	repo.spring.io/milestone
Release Candidate	`7.2.0-RC1`	Feature-complete, API frozen	repo.spring.io/milestone
GA	`7.2.0`	Production-ready	**Maven Central**
Patch	`7.2.1`	Bugfix only, no pre-releases	**Maven Central**

Rule: Patches skip all pre-release stages → `7.2.1-SNAPSHOT` becomes `7.2.1` directly.
There is no such thing as `7.2.1-M1`.

SNAPSHOT: Always the Latest Build

`7.2.0-SNAPSHOT` is **not a fixed artifact** — it means "latest successful CI build of 7.2.0"

- Resolves to **different bytecode** tomorrow than it does today
- Maven re-checks the remote once per day by default; Gradle has a configurable TTL
- **Never on Maven Central** — Central enforces immutability; SNAPSHOTs are mutable by definition
- Requires explicit repository config to use:

```
<repositories>
  <repository>
    <id>spring-snapshots</id>
    <url>https://repo.spring.io/snapshot</url>
    <snapshots><enabled>true</enabled></snapshots>
  </repository>
</repositories>
```

> **Who should use SNAPSHOTs:** Contributors testing unreleased changes. Not production applications.

Milestones and Release Candidates

Milestone (M1, M2...): Fixed immutable artifact. API still in flux.

"Here is a stake in the ground for early adopter feedback."

RC (RC1, RC2...): Feature-complete, API frozen.

"We believe this could be GA if no blocking bugs are found."

	Milestone	Release Candidate
Features complete?	No	**Yes**
API stable?	Stabilizing	**Frozen**
Breaking changes?	Possible	No
Purpose	Shape API feedback	Bug hunting
Typical cycle time	Weeks–months	Days–weeks
Repository	repo.spring.io/milestone	repo.spring.io/milestone

Spring 6.0 used **M1 through M5** before RC1 — more complex cycles take more milestones.

GA and Patches: The Stable World

GA — General Availability

- Plain version string: `7.2.0` (no suffix)
- The **only** artifact type published to Maven Central
- Immutable forever — a bug in `7.2.0` is addressed in `7.2.1`, not by replacing `7.2.0`
- Historical note: Spring 5.x used `.RELEASE` suffix (`5.3.39.RELEASE`). Spring 6.0 dropped it.

Patch releases

- Increment the third number: `7.2.1`, `7.2.2`...
- **Bugfixes and security patches only** — no new APIs, no feature additions
- Go **directly** from SNAPSHOT to GA — no milestones, no RCs:

```
7.2.1-SNAPSHOT  ──────────────────────►  7.2.1 (GA)
```

- Active lines release patches approximately monthly — coordinated across all active branches

Release Trains and the BOM

Spring releases an **ecosystem**, not a single library. Spring Boot is the coordinator.

```
Spring Boot 3.3.0 (the conductor)
    │
    ├── Spring Framework  6.1.8
    ├── Spring Security   6.3.0
    ├── Spring Data       3.3.0 (Quince)
    ├── Spring Batch      5.1.0
    ├── Micrometer        1.13.0
    ├── Reactor Core      3.6.6
    └── Tomcat            10.1.x
```

- **BOM** (Bill of Materials): a Maven POM that pins compatible versions across the ecosystem
- `"We're on Spring Boot 3.3"` = the entire constellation above, coordinated
- `"Upgrade to Spring Boot 3.4"` = upgrading the **whole train**, not one artifact

Same Concept, Different Vocabulary

All major OSS projects have the same underlying stages — the labels differ by heritage.

Stage	Spring	Quarkus	Kafka	Kubernetes
Mutable CI	SNAPSHOT	—	SNAPSHOT	dev build
Early feedback	M1	Alpha1	— *(skips to RC)*	alpha.1
API stabilizing	M2/M3	Beta1	—	beta.0
Feature-complete	RC1	**CR1**	rc1	rc.0
Stable	`7.2.0`	`3.27.0` *(was* `.Final` *)*	`3.8.0`	`v1.36.0`

`CR` = Candidate Release (JBoss/Red Hat term for RC)

Kafka skips milestone stages entirely — RC-only pre-release

Kubernetes uses semver pre-release identifiers, not Maven version qualifiers

What Makes Each Project Distinctive

Apache Kafka

- Only RC stage — no milestones, no alpha/beta
- Requires a **PMC community vote** to publish — any committer can veto with a -1
- Uses `trunk` instead of `main` (SVN heritage)

Quarkus

- Explicit **LTS designations** every ~6 months; non-LTS minors supported for ~4 weeks
- Ships ~one new minor per month (much faster cadence than Spring)
- Emergency branches for critical CVEs: `3.20.2-emergency`

Kubernetes

- **N-2 support**: exactly 3 minors simultaneously — when 1.36 ships, 1.33 is EOL immediately
- **Time-boxed**: 3 minors per year on a calendar. Features that miss the window wait.
- Patches on the 28th of every month across all 3 supported minors

The Vocabulary Reflects the Heritage

Project	Heritage	Vocabulary
Spring	Pivotal/VMware · JVM · Maven	SNAPSHOT → M1 → RC1 → GA
Quarkus / Hibernate	Red Hat · JBoss AS (acquired 2006)	Alpha → Beta → **CR** → **Final**
Apache Kafka / Camel	Apache Software Foundation · SVN era	RC1 → GA (via community vote)
Kubernetes	Linux kernel · Go · CNCF	alpha.1 → beta.0 → **rc.0** → stable

> When you see `CR1` and `Final` → you are in the JBoss/Red Hat lineage
> When you see `trunk` and a PMC vote → you are in the ASF lineage
> When you see `alpha.1` as semver pre-release → you are in the systems/Kubernetes lineage

The mental model: understand the project's heritage and you can predict its conventions.

What You Now Know

You see...	It means...	Where
`7.2.0-SNAPSHOT`	Mutable CI build — changes daily	snapshot repo
`7.2.0-M1`	Fixed pre-release, API in flux	milestone repo
`7.2.0-RC1`	Feature-complete, API frozen	milestone repo
`7.2.0`	Production-ready	**Maven Central**
`7.2.1`	Bugfix patch, no pre-release stages	**Maven Central**

- **Branches:** `N.N.x` owns all patches for that minor · `main` = next gen, not current stable
- **Fixes:** commit to oldest branch, merge forward · never backward
- **Boot:** one Boot version = the whole ecosystem · BOM pins all the rest
- **Other projects:** same concepts, different vocabulary — heritage predicts conventions

📄 Reference doc: `docs/oss-release-strategy.md`
📊 Diagrams: `docs/diagrams.md`
🔗 Support dates: https://spring.io/projects/spring-framework#support